



Universidad Nacional Autónoma de México
Facultad de Ingeniería

Materia: Organización Y Arquitectura de
Computadoras

Profesor: M.I. José Antonio de Jesús Arredondo Garza

Práctica 1
Raíz Cuadrada

No. de Cuenta	Nombre
320068234	Ayala Hernández María Fernanda
320335163	Salazar Islas Luis Daniel
320078282	Tepal Briseño Hansel Yael
320223822	Ugartechea González Luis Antonio

Fecha de entrega: <fecha_entrega>

Calificación: _____

1 Objetivo

En esta práctica se implementó una función para calcular la raíz entera de un número mediante la resta sucesiva de números impares. La idea principal es que la suma de los primeros n números impares es n^2 , por lo que al restar $1, 3, 5, 7, \dots$ se puede contar cuántas restas fueron necesarias para llegar a cero o a un valor menor.

2 Desarrollo

El desarrollo de la práctica consistió en implementar la función de raíz entera en dos lenguajes: MicroPython y ensamblador ARM. La función toma un número entero positivo y devuelve su raíz entera.

El algoritmo funciona de la siguiente manera:

- Inicializa el resto con el valor original.
- Resta secuencialmente $1, 3, 5, 7, \dots$
- Cada resta exitosa representa una unidad de la raíz.
- Cuando el resto ya no alcanza al siguiente número impar, se termina y se devuelve el contador.

Por ejemplo, para $x = 25$:

$$25 - 1 - 3 - 5 - 7 - 9 = 0$$

Se realizaron 5 restas, por lo que $\lfloor \sqrt{25} \rfloor = 5$.

3 Implementación en Python

La implementación directa en MicroPython permite probar la lógica de manera sencilla. La función `py_odd_sqrt` recibe un número entero y devuelve su raíz entera.

```
1 def py_odd_sqrt(x):
2     remainder = x
3     odd_number = 1
4     root = 0
5
6     while remainder >= odd_number:
7         remainder -= odd_number
8         odd_number += 2
9         root += 1
10
11     return root
```

4 Implementación en ARM

La función ARM realiza el mismo procedimiento en lenguaje ensamblador, usando registros para llevar el valor restante, el número impar actual y el contador de la raíz. La lógica es equivalente a la versión en Python, pero optimizada para ejecutarse directamente en la placa con la función `odd_sqrt`.

```
1 @micropython.asm_thumb
2 def odd_sqrt(r0):
3     mov(r4, r0) # Remaining value
4     mov(r1, 1) # Current odd number
5     mov(r2, 0) # Integer square root
6
7     label(LOOP)
8     cmp(r1, r4) # Check if the current odd number is greater than
9                 the remaining value
10    bhi(DONE)
11
12    sub(r4, r4, r1) # Subtract the current odd number from the
13                   remaining value
14    add(r1, r1, 2) # Increment the current odd number by 2
15    add(r2, r2, 1) # Counts one successful subtraction, which
16                   corresponds to the integer square root
17    b(LOOP)
18
19    label(DONE)
20    mov(r0, r2)
```

Listing 2: Implementación principal en ARM

5 Pruebas y Validación

Esta sección de código ejecuta una serie de pruebas en donde se compara la salida de el calculo obtenido para la solución en Python y la solución en ARM con una serie de valores esperados. Se imprimen los resultados de cada prueba, indicando si la prueba pasó o falló.

```
1 tests = [
2     (0, 0),
3     (1, 1),
4     (2, 1),
5     (3, 1),
```

```

6      (4, 2),
7      (5, 2),
8      (8, 2),
9      (9, 3),
10     (10, 3),
11     (24, 4),
12     (25, 5),
13     (35, 5),
14     (36, 6),
15     (50, 7),
16     (99, 9),
17     (100, 10),
18     (1000, 31),
19 ]
20
21 print("=" * 42)
22 print("  INTEGER SQUARE ROOT - ODD NUMBERS")
23 print("=" * 42)
24 print("\n[ARM Assembly]")
25
26 for x, expected in tests:
27     result = odd_sqrt(x)
28     if result == expected:
29         print("  PASS | input: {:>6} | sqrt: {:>4}".format(x, result
30         ))
31     else:
32         print("  FAIL | input: {:>6} | expected: {:>4} | got: {:>4}"
33         .format(x, expected, result))
34
35 print("\n[MicroPython]")
36
37 for x, expected in tests:
38     result = py_odd_sqrt(x)
39     if result == expected:
40         print("  PASS | input: {:>6} | sqrt: {:>4}".format(x, result
41         ))
42     else:
43         print("  FAIL | input: {:>6} | expected: {:>4} | got: {:>4}"
44         .format(x, expected, result))
45
46 print("\n" + "=" * 42)
47 print("                                DONE")
48 print("=" * 42)

```

Listing 3: Validación de casos de prueba en Python

6 Ejecución en la Raspberry Pi Pico

El archivo principal se puede cargar a la placa con el siguiente comando desde la carpeta raíz del proyecto:

```
1 ./upload.sh practica_01/source/practica_01.py
```

El contenido del script que realiza esta ejecución es el siguiente:

```
1  #!/usr/bin/env bash
2
3  set -euo pipefail
4
5  script="${1:?Usage: ./upload.sh SCRIPT.py [DEVICE]}"
6  device="${2:-/dev/ttyACM0}"
7
8  if [[ ! -f "$script" ]]; then
9      echo "File not found: $script" >&2
10     exit 1
11 fi
12
13 if fuser "$device" >/dev/null 2>&1; then
14     echo "Port $device is busy. Stopping the process using it..." >&2
15     fuser -k "$device" >/dev/null 2>&1 || true
16 fi
17
18 mpremote connect "$device" fs cp "$script" :main.py
19 mpremote connect "$device" run "$script"
```

Listing 4: Script de carga a la Pico

Este script:

- valida que el archivo exista,
- libera el puerto si está ocupado,
- copia el programa a la Pico como :main.py,
- y lo ejecuta inmediatamente con mpremote.

7 Resultados

La salida real obtenida al ejecutar el programa se muestra a continuación:

```
1 cp practica_01/source/practica_01.py :main.py
2 Up to date: main.py
3 =====
```

```

4  INTEGER SQUARE ROOT - ODD NUMBERS
5  =====
6
7  [ARM Assembly]
8  PASS | input:      0 | sqrt:    0
9  PASS | input:      1 | sqrt:    1
10 PASS | input:      2 | sqrt:    1
11 PASS | input:      3 | sqrt:    1
12 PASS | input:      4 | sqrt:    2
13 PASS | input:      5 | sqrt:    2
14 PASS | input:      8 | sqrt:    2
15 PASS | input:      9 | sqrt:    3
16 PASS | input:     10 | sqrt:    3
17 PASS | input:     24 | sqrt:    4
18 PASS | input:     25 | sqrt:    5
19 PASS | input:     35 | sqrt:    5
20 PASS | input:     36 | sqrt:    6
21 PASS | input:     50 | sqrt:    7
22 PASS | input:     99 | sqrt:    9
23 PASS | input:    100 | sqrt:   10
24 PASS | input:   1000 | sqrt:   31
25
26 [MicroPython]
27 PASS | input:      0 | sqrt:    0
28 PASS | input:      1 | sqrt:    1
29 PASS | input:      2 | sqrt:    1
30 PASS | input:      3 | sqrt:    1
31 PASS | input:      4 | sqrt:    2
32 PASS | input:      5 | sqrt:    2
33 PASS | input:      8 | sqrt:    2
34 PASS | input:      9 | sqrt:    3
35 PASS | input:     10 | sqrt:    3
36 PASS | input:     24 | sqrt:    4
37 PASS | input:     25 | sqrt:    5
38 PASS | input:     35 | sqrt:    5
39 PASS | input:     36 | sqrt:    6
40 PASS | input:     50 | sqrt:    7
41 PASS | input:     99 | sqrt:    9
42 PASS | input:    100 | sqrt:   10
43 PASS | input:   1000 | sqrt:   31
44
45 =====
46                      DONE
47 =====

```

Listing 5: Salida generada por la práctica

8 Conclusión

La práctica demostró que la raíz entera puede obtenerse de manera eficiente a partir de la suma de números impares. La versión en Python permite entender claramente el procedimiento, mientras que la versión en ARM confirma la viabilidad de la misma lógica en un entorno embebido.